ПРАКТИЧНА РОБОТА №5

Тема: Flexbox, FlatList та оновлення списку жестом
Мета: Переробити екран подій для екранів різної ширини, вивести дані через FlatList і реалізувати локальне оновлення списку жестом потягування.

ХІД РОБОТИ

1. Додано `additionalEvents` із двох навчальних подій із власними `id` у `src/data/events.ts`.

```
export const additionalEvents: EventItem[] = [
    {
        id: 'refresh-1',
        title: 'Майстер-клас із цифрового дизайну',
        date: '18.10.2026, 12:00',
        location: 'Комп'ютерна лабораторія',
        category: 'Навчання',
        description: 'Практичне заняття зі створення афіш та візуальних матеріалів для студентських ініціатив.'
    },
    {
        id: 'refresh-2',
        title: 'Осінній турнір із настільного тенісу',
        date: '22.10.2026, 15:30',
        location: 'Спортивна зала',
        category: 'Спорт',
        description: 'Відкритий турнір для студентів і викладачів коледжу. Реєстрація учасників перед початком змагань.'
    },
];
```

2. Список на головному екрані переведено з ScrollView на один FlatList. Банер, заголовок, фільтри, лічильник і форма розташовані в ListHeaderComponent; порожній список показує ListEmptyComponent.

```
<FlatList
  key={columns}
  data={visibleEvents}
  renderItem={renderEvent}
  keyExtractor={keyExtractor}
  extraData={favoriteIds}
  numColumns={columns}
  columnWrapperStyle={columns === 2 ? styles.columnWrapper : undefined}
  ListHeaderComponent={listHeader}
  ListEmptyComponent={(
    <Text style={styles.emptyText}>У цій категорії поки немає подій</Text>
  )}
  contentContainerStyle={styles.content}
  keyboardShouldPersistTaps="handled"
  refreshing={refreshing}
  onRefresh={refreshEvents}
/>
```

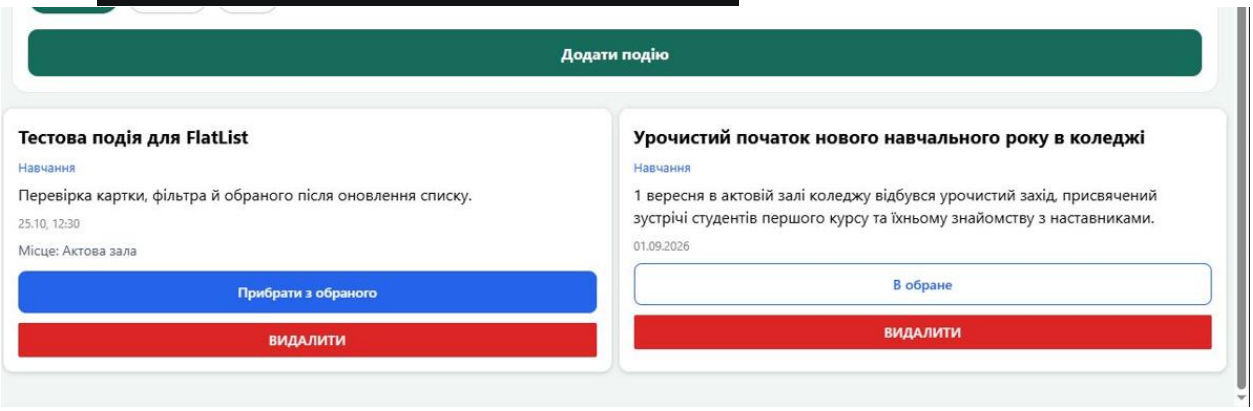
```
const listHeader = (
  <View>
    <Image
      source={require('../../assets/images/college-events.jpg')}
      style={styles.banner}
      accessible
      accessibilityLabel="Студенти спілкуються у дворі коледжу біля будівлі"
      resizeMode="cover"
    />
    <Text style={styles.title}>Події коледжу</Text>

    <View style={styles.filters}>
      {categories.map((item) => {
        const isActive = category === item;
        return (
          <Pressable
            key={item}
            style={[styles.filterButton, isActive && styles.activeFilter]}
            onPress={() => setCategory(item)}
            accessibilityRole="button"
            accessibilityLabel={`Фільтр: ${item}`}
            accessibilityState={{ selected: isActive }}
          >
            <Text style={[styles.filterText, isActive && styles.activeFilterText]}>
              {item}
            </Text>
          </Pressable>
        );
      })}
    </View>

    <Text style={styles.favoriteCount}>Обрано: {favoriteIds.length}</Text>
    {filterNotice !== '' && <Text style={styles.notice}>{filterNotice}</Text>}
    <EventForm onAdd={addEvent} />
  </View>
);
```

3. Для кожного елемента використано стабільний ключ `event.id`, а `favoriteIds` передається в `extraData`, тому позначка картки оновлюється після натискання.
4. Ширина визначається через `useWindowDimensions()`: до 700 включно — одна колонка, від 700 — дві. При зміні числа колонок список перемонтовується через `key={columns}`; стан подій і обраного залишається у батьківському екрані.

```
const { width } = useWindowDimensions();
const columns = width >= 700 ? 2 : 1;
```



5. Картки використовують `flex: 1`, переносять довгі назви й не мають фіксованої висоти. `EventCard` обгорнутий у `memo`; стабільні callback-и дають змогу не перемальовувати незмінені картки.

```
const renderEvent = useCallback(({ item }: ListRenderItemInfo<EventItem>) => (
  <EventCard
    event={item}
    onDelete={deleteEvent}
    isFavorite={favoriteIds.includes(item.id)}
    onToggleFavorite={toggleFavorite}
  />
), [deleteEvent, favoriteIds, toggleFavorite]);
```

6. Pull-to-refresh показує індикатор, блокує паралельні оновлення й додає тільки події з новими `id`. Користувацькі події та `favoriteIds` залишаються в стані.

```
const refreshEvents = useCallback(() => {
  if (refreshInProgress.current) {
    return;
  }

  refreshInProgress.current = true;
  setRefreshing(true);

  setTimeout(() => {
    setEvents((previous) => {
      const existingIds = new Set(previous.map((event) => event.id));
      const newEvents = additionalEvents.filter((event) => !existingIds.has(event.id));
      return newEvents.length > 0 ? [...previous, ...newEvents] : previous;
    });
    setRefreshing(false);
    refreshInProgress.current = false;
  }, 800);
}, []);
```

Контрольні питання

1. Початкове значення — `column`, тому головна вісь вертикальна.
2. `justifyContent` розподіляє елементи вздовж головної осі: у `column` — вертикально, у `row` — горизонтально. Вісь `alignItems` при цьому стає поперечною..
3. Ширина вікна напряму описує доступний простір і змінюється після повороту, зміни розміру вікна чи на складаних пристроях. Назва моделі не враховує такі зміни та не завжди точно описує реальну ширину інтерфейсу.
4. `ScrollView` монтує всі дочірні елементи одразу. `FlatList` створює рядки у вікні прокручування й підвантажує сусідні рядки в міру руху списком.
5. `keyExtractor` дає кожній події постійну унікальну ідентичність для повторного впорядкування та фільтрації. `extraData` змушує `FlatList` переоцінити рядки, якщо змінилася їхня залежність поза `data`, тут — список обраного.
6. Рядок поза видимою областю може бути демонтований, і його локальний стан зникне. Стан, який має переживати прокручування,

слід зберігати у батьківському екрані або окремому сховищі й передавати картці через props.

7. Перед додаванням порівняти `id` нових записів із поточними, наприклад через `Set`, і додати тільки відсутні. Додатково можна не запускати паралельне оновлення, поки попереднє ще триває.